

DATA ENGINEERING

SQL Interview Prep Guide

Core SQL concepts every data engineer needs — from joins and keys to window functions, indexing, normalization, and query execution order.

SQL Execution Order · Joins · Primary vs Foreign Keys · WHERE vs HAVING · Normalization · Indexes · Window Functions · Subqueries vs CTEs · ACID & Transactions · UNION vs UNION ALL · DELETE vs TRUNCATE vs DROP · SQL vs NoSQL

Table of Contents

1. How SQL Queries Actually Execute
2. SQL Joins Explained
3. Primary Key vs. Foreign Key
4. WHERE vs. HAVING
5. Database Normalization (1NF–3NF)
6. Indexes — How and Why They Speed Up Queries
7. Window Functions
8. Subqueries vs. CTEs (WITH clause)
9. ACID Properties & Transactions
10. UNION vs. UNION ALL
11. DELETE vs. TRUNCATE vs. DROP
12. SQL vs. NoSQL

How SQL Queries Actually Execute

One of the most common SQL interview questions: *"What order does SQL execute clauses in?"* The order you **write** a query is not the order it **executes** in — this trips up almost everyone at first.

Written Order (what you type)

```
SELECT    column_list
FROM      table
JOIN      other_table ON ...
WHERE     condition
GROUP BY  column
HAVING    group_condition
ORDER BY  column
LIMIT     n
```

Logical Execution Order (what actually happens)

1. FROM → identify the source table(s)
2. JOIN → combine rows from multiple tables
3. WHERE → filter individual rows
4. GROUP BY → group the remaining rows
5. HAVING → filter the groups
6. SELECT → compute the output columns / expressions
7. DISTINCT → remove duplicate rows
8. ORDER BY → sort the result
9. LIMIT/OFFSET → restrict the number of rows returned

WHY THIS MATTERS

Because **WHERE** runs before **SELECT**, you cannot reference a column alias defined in **SELECT** inside a **WHERE** clause. Because **HAVING** runs after **GROUP BY**, it can filter on aggregate functions like **COUNT(*)** — **WHERE** cannot.

Example

```
SELECT department, COUNT(*) AS emp_count
FROM employees
WHERE salary > 5000
GROUP BY department
HAVING COUNT(*) > 3
ORDER BY emp_count DESC;
```

Step	What Happens
FROM	Read the employees table
WHERE	Keep only rows where salary > 5000

GROUP BY	Group remaining rows by department
HAVING	Keep only groups with more than 3 employees
SELECT	Compute department and COUNT(*)
ORDER BY	Sort by emp_count descending

Common Mistake

```
-- This fails: alias 'emp_count' doesn't exist yet at WHERE-time
SELECT department, COUNT(*) AS emp_count
FROM employees
WHERE emp_count > 3
GROUP BY department;

-- Correct: use HAVING for aggregate filtering
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 3;
```

Interview Q&A

Q1. What's the difference between the written order and the execution order of a SQL query?

SQL is written SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY, but executes FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. The engine identifies data first, filters and groups it, then computes the final output columns.

Q2. Why can't you use a column alias in WHERE?

Because WHERE executes before SELECT, so the alias hasn't been computed yet. Most databases allow aliases in ORDER BY and GROUP BY since those run after SELECT (behavior varies slightly by engine).

Q3. Why does HAVING exist if WHERE already filters?

WHERE filters individual rows before grouping and cannot reference aggregate functions. HAVING filters groups after aggregation, so it can use COUNT(), SUM(), AVG(), etc.

INTERVIEW SUMMARY

- SQL's logical execution order is: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.
- WHERE filters rows before grouping; HAVING filters groups after aggregation.
- Column aliases defined in SELECT aren't available in WHERE, because WHERE runs first.

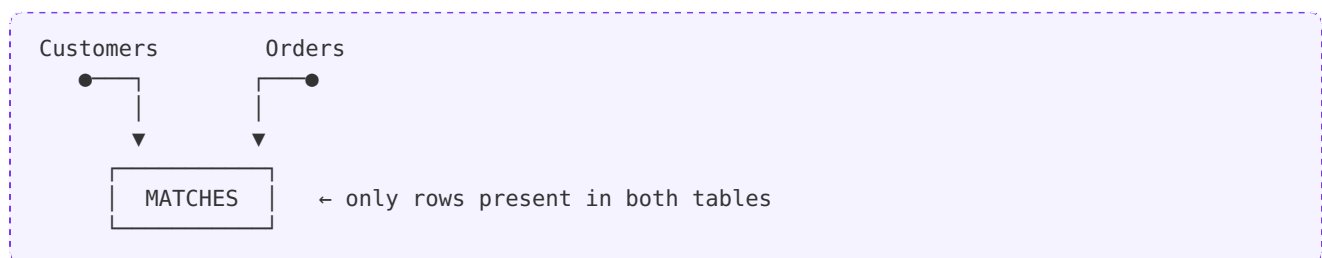
SQL Joins Explained

Joins combine rows from two or more tables based on a related column. Choosing the wrong join is one of the most common sources of bugs (and interview trip-ups).

INNER JOIN

Returns only rows that have matching values in **both** tables.

```
SELECT c.name, o.order_id
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;
```



LEFT JOIN (LEFT OUTER JOIN)

Returns **all** rows from the left table, plus matched rows from the right table. Unmatched right-side columns are **NULL**.

```
SELECT c.name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
-- Customers with no orders still appear, with o.order_id = NULL
```

RIGHT JOIN (RIGHT OUTER JOIN)

Mirror of LEFT JOIN — returns all rows from the right table, plus matched rows from the left. Rarely used in practice; most people rewrite it as a LEFT JOIN with tables swapped.

FULL OUTER JOIN

Returns all rows from **both** tables — matched rows are combined, unmatched rows from either side are padded with **NULL**.

```
SELECT c.name, o.order_id
FROM customers c
FULL OUTER JOIN orders o ON c.customer_id = o.customer_id;
```

CROSS JOIN

Returns the **Cartesian product** — every row from table A paired with every row from table B. No ON condition. If A has 5 rows and B has 3 rows, the result has 15 rows.

```
SELECT s.size, c.color
FROM sizes s
CROSS JOIN colors c;
```

SELF JOIN

A table joined to itself — useful for hierarchical or comparative data (e.g. employees and their managers, both stored in the same table).

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id;
```

Visual Summary

INNER JOIN
A $\cap$ B


LEFT JOIN
All of A
(+ NULLs on B)


FULL OUTER JOIN
All of A + All of B
(+ NULLs on both sides)


Comparison Table

Join Type	Returns	Unmatched Rows
INNER JOIN	Only matching rows	Excluded from both sides
LEFT JOIN	All left rows + matches	Right side filled with NULL
RIGHT JOIN	All right rows + matches	Left side filled with NULL
FULL OUTER JOIN	All rows from both	NULL on whichever side has no match
CROSS JOIN	Every combination (Cartesian product)	N/A — no matching condition

Finding Non-Matching Rows ("Anti-Join")

A common interview task: find customers with **no** orders.

```
SELECT c.name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.customer_id IS NULL;
```

Interview Q&A

Q1. Difference between INNER JOIN and LEFT JOIN?

INNER JOIN returns only rows with matches in both tables. LEFT JOIN returns all rows from the left table regardless of a match, filling unmatched right-side columns with NULL.

Q2. How do you find rows in table A with no match in table B?

LEFT JOIN A to B, then filter WHERE B's join key IS NULL — this is the "anti-join" pattern.

Q3. When would you use a CROSS JOIN?

When you deliberately need every combination of two sets — e.g. generating all size/color product variants, or building a date-spine calendar table.

Q4. What is a self join used for?

Comparing rows within the same table — e.g. employee-to-manager relationships, or finding duplicate records.

INTERVIEW SUMMARY

- INNER JOIN → only matches. LEFT/RIGHT JOIN → all rows from one side + matches. FULL OUTER JOIN → all rows from both sides.
- CROSS JOIN produces a Cartesian product — no join condition needed.
- A self join relates a table to itself, useful for hierarchies.
- LEFT JOIN + `WHERE right.key IS NULL` finds unmatched rows (anti-join).

Primary Key vs. Foreign Key

Keys enforce data integrity and define the relationships between tables in a relational database.

Primary Key

A column (or set of columns) that **uniquely identifies** each row in a table.

- Must be **unique** across all rows
- Cannot be **NULL**
- Only **one** primary key per table (though it can span multiple columns — a composite key)
- Automatically indexed by most database engines

```
CREATE TABLE employees (  
  employee_id INT PRIMARY KEY,  
  name VARCHAR(100),  
  department VARCHAR(50)  
);
```

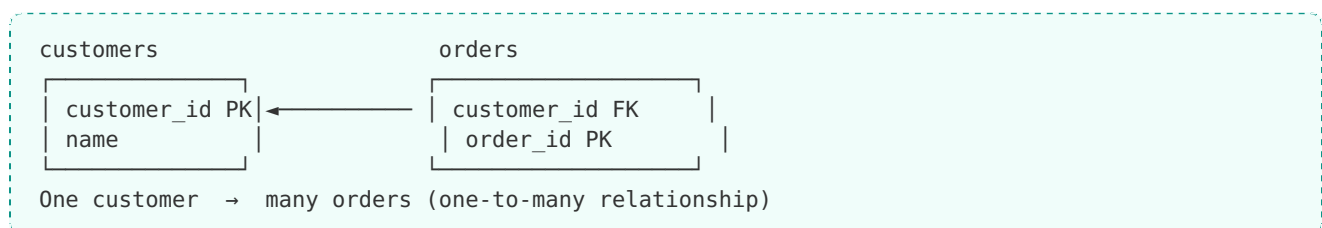
Foreign Key

A column that references the **primary key** of another table, establishing a relationship between the two.

- Enforces **referential integrity** — you can't insert a foreign key value that doesn't exist in the referenced table
- Can contain **NULL** (unless explicitly constrained otherwise)
- A table can have **multiple** foreign keys
- Values can repeat (many rows can reference the same parent row)

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_id INT,  
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

Visual Relationship



Comparison Table

Feature	Primary Key	Foreign Key
Purpose	Uniquely identifies each row	Links to a primary key in another table
Uniqueness	Must be unique	Can have duplicate values
NULL Allowed	No	Yes (unless restricted)
Count per Table	One (can be composite)	Multiple allowed
Indexing	Automatically indexed	Often manually indexed for join performance

Composite Key

A primary key made up of **more than one column** — used when no single column is unique on its own.

```
CREATE TABLE order_items (  
  order_id INT,  
  product_id INT,  
  quantity INT,  
  PRIMARY KEY (order_id, product_id)  
);
```

Interview Q&A

Q1. Can a foreign key be NULL?

Yes, unless explicitly marked NOT NULL — meaning the row simply has no related parent record.

Q2. Can a table have more than one primary key?

No — only one primary key per table, but it can be composite (span multiple columns).

Q3. What happens if you try to insert a foreign key value with no matching primary key?

The database rejects the insert with a referential integrity / foreign key constraint violation error.

Q4. What is referential integrity?

The guarantee that foreign key values always correspond to an existing primary key value in the referenced table — preventing "orphaned" records.

INTERVIEW SUMMARY

- A Primary Key uniquely identifies each row, is never NULL, and one exists per table.

- A Foreign Key references another table's primary key, enforcing referential integrity; multiple can exist per table and values may repeat.
- Together they model relationships (one-to-many, many-to-many via junction tables).

WHERE vs. HAVING

Both filter data, but at **different stages** of query execution — this is one of the most frequently asked basic SQL questions.

WHERE

Filters **individual rows** before any grouping happens. Cannot use aggregate functions like `COUNT()`, `SUM()`, `AVG()`.

```
SELECT name, salary
FROM employees
WHERE salary > 5000;
```

HAVING

Filters **groups** after `GROUP BY` has run. Designed specifically to filter on aggregate functions.

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 6000;
```

Side by Side

Raw Rows → [WHERE filters rows] → GROUP BY → [HAVING filters groups] → SELECT output

Feature	WHERE	HAVING
Filters	Individual rows	Aggregated groups
Runs	Before GROUP BY	After GROUP BY
Aggregate Functions	Not allowed	Allowed (COUNT, SUM, AVG...)
Used Without GROUP BY?	Yes, commonly	Rare, but legal (treats whole result as one group)
Performance	Generally faster — filters early, reduces rows before grouping	Runs after aggregation, so it processes more data first

Using Both Together

```
SELECT department, COUNT(*) AS emp_count
FROM employees
WHERE hire_date >= '2023-01-01' -- filter rows first (cheap)
```

```
GROUP BY department
HAVING COUNT(*) > 5;           -- then filter groups (on aggregate)
```

PERFORMANCE TIP

Always push filters into WHERE whenever possible — it reduces the number of rows before the (more expensive) grouping and aggregation step. Only use HAVING for conditions that genuinely depend on an aggregate value.

Interview Q&A

Q1. Can you use HAVING without GROUP BY?

Yes — the entire result set is treated as a single group, but this is uncommon in practice.

Q2. Why can't WHERE filter on COUNT(*)?

Because WHERE executes before grouping/aggregation happens, so aggregate values don't exist yet at that stage.

Q3. Which is more efficient, filtering in WHERE or HAVING?

WHERE is generally more efficient when the condition doesn't depend on an aggregate, since it reduces the row count before the costlier grouping step.

INTERVIEW SUMMARY

- WHERE filters rows before grouping; cannot reference aggregate functions.
- HAVING filters groups after GROUP BY; built for aggregate conditions.
- Use WHERE to cut down rows early, and HAVING only for aggregate-based conditions.

Database Normalization (1NF-3NF)

Normalization is the process of organizing tables to **reduce data redundancy** and avoid update, insert, and delete anomalies. It works through a series of "normal forms," each building on the last.

Unnormalized Example

Orders (unnormalized)

OrderID	CustomerName	Products
1	Ali	Laptop, Mouse, Keyboard

← multiple values in one cell

1NF — First Normal Form

Eliminate repeating groups: each cell must hold a single (atomic) value, and each row must be unique.

Orders (1NF)

OrderID	CustomerName	Product
1	Ali	Laptop
1	Ali	Mouse
1	Ali	Keyboard

2NF — Second Normal Form

Must already be in 1NF, plus: every non-key column must depend on the **whole** primary key (relevant when the key is composite) — no **partial dependency**.

Before 2NF (CustomerName depends only on OrderID, not on Product)

OrderID + Product (composite key) | CustomerName | Price

After 2NF — split into two tables:

Orders: OrderID | CustomerName

OrderItems: OrderID | Product | Price

3NF — Third Normal Form

Must already be in 2NF, plus: no **transitive dependency** — non-key columns must depend only on the primary key, not on another non-key column.

Before 3NF (ZipCode determines City — a transitive dependency)

EmployeeID | ZipCode | City

After 3NF — split into two tables:

Employees: EmployeeID | ZipCode
ZipCodes: ZipCode | City

Comparison Table

Normal Form	Rule	Fixes
1NF	Atomic values, unique rows, no repeating groups	Multiple values crammed into one column
2NF	1NF + no partial dependency on a composite key	Data duplicated across rows sharing part of the key
3NF	2NF + no transitive dependency between non-key columns	Redundant data derivable from another non-key column

Why Normalize?

Reduces redundancy

Prevents update anomalies

Prevents insert anomalies

Prevents delete anomalies

Smaller storage footprint

The Trade-off: Denormalization

Highly normalized schemas require more joins to reconstruct data, which can hurt read performance. In analytics/data-warehouse settings, tables are often **deliberately denormalized** (e.g. star schema) to optimize for fast reads over write efficiency.

Normalized	Denormalized
Less redundancy, smaller storage	More redundancy, larger storage
Slower reads (more joins)	Faster reads (fewer joins)
Safer writes/updates	Risk of update anomalies
Common in OLTP systems	Common in OLAP / data warehouses

Interview Q&A

Q1. What is normalization?

The process of structuring a database to reduce redundancy and prevent update/insert/delete anomalies by organizing data into related tables.

Q2. What's a transitive dependency?

When a non-key column depends on another non-key column rather than directly on the primary key (e.g. City depends on ZipCode, not on EmployeeID).

Q3. When would you denormalize a database?

In read-heavy analytical systems (data warehouses, reporting) where join performance matters more than write-time redundancy control — e.g. star schemas in BI.

INTERVIEW SUMMARY

- 1NF: atomic values, no repeating groups. 2NF: no partial dependency on a composite key. 3NF: no transitive dependency between non-key columns.
- Normalization reduces redundancy and anomalies but increases the number of joins needed to read data.
- OLTP systems favor normalization; OLAP/data warehouses often favor denormalization for read speed.

Indexes — How and Why They Speed Up Queries

An index is a separate data structure that lets the database find rows **without scanning the entire table** — similar to an index at the back of a book.

Without an Index

```
SELECT * FROM employees WHERE employee_id = 4821;
```

Full Table Scan:

Row 1 → check → no match

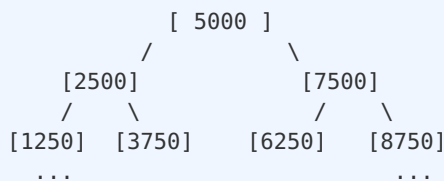
Row 2 → check → no match

Row 3 → check → no match

...

Row 4821 → check → MATCH (scanned thousands of rows)

With an Index (B-Tree)



Lookup for 4821 walks the tree in $O(\log n)$ instead of scanning every row → dramatically fewer comparisons on large tables.

Creating an Index

```
CREATE INDEX idx_employee_id ON employees(employee_id);

-- Composite index (order matters!)
CREATE INDEX idx_dept_salary ON employees(department, salary);
```

Types of Indexes

- **Clustered Index** — determines the physical order rows are stored on disk. Only one per table (often the primary key).
- **Non-Clustered Index** — a separate structure that points back to the actual row location. A table can have many.
- **Unique Index** — enforces uniqueness in addition to speeding up lookups (automatically created for primary keys).
- **Composite Index** — spans multiple columns; column order affects which queries can use it.

The Trade-off

Benefit	Cost
Much faster SELECT / WHERE / JOIN / ORDER BY on indexed columns	Slower INSERT / UPDATE / DELETE — the index must be updated too
Enables efficient uniqueness enforcement	Extra disk space to store the index structure
Speeds up sorting (ORDER BY on indexed column)	Too many indexes can hurt write-heavy workloads

When an Index Won't Help

- Using a function on the indexed column: `WHERE YEAR(hire_date) = 2024` (prevents index use unless a functional index exists)
- Leading wildcard search: `WHERE name LIKE '%son'` (can't use a standard B-Tree index efficiently)
- Low-cardinality columns (e.g. a boolean "is_active" flag with only 2 distinct values) — the optimizer may prefer a full scan anyway
- Small tables — a full scan can be cheaper than the overhead of using an index

Checking If an Index Is Used

```
EXPLAIN SELECT * FROM employees WHERE employee_id = 4821;  
-- Look for "Index Scan" / "Index Seek" vs. "Seq Scan" / "Table Scan" in the plan
```

Interview Q&A

Q1. What is an index and why use one?

A separate data structure (commonly a B-Tree) that allows the database to locate rows without scanning the whole table, dramatically speeding up reads on large tables.

Q2. Difference between clustered and non-clustered index?

A clustered index determines the physical storage order of rows (only one per table). A non-clustered index is a separate lookup structure pointing to row locations (a table can have several).

Q3. Why not index every column?

Every index adds overhead to INSERT/UPDATE/DELETE operations (the index must be maintained) and consumes extra storage — indexes should be added deliberately based on query patterns.

Q4. Does column order matter in a composite index?

Yes — a composite index on (department, salary) can efficiently serve queries filtering on department alone, or department+salary together, but generally not on salary alone.

INTERVIEW SUMMARY

- Indexes (typically B-Trees) let the database avoid full table scans, turning $O(n)$ lookups into roughly $O(\log n)$.
- Clustered indexes control physical row order (one per table); non-clustered indexes are separate lookup structures (many per table).
- Indexes speed up reads but slow down writes and use extra storage — apply them deliberately based on real query patterns.

Window Functions

Window functions perform a calculation across a set of related rows ("a window") **without collapsing them into a single row** — unlike `GROUP BY`, every original row is preserved.

Basic Syntax

```
function_name(column) OVER (
  PARTITION BY column  -- optional: split into groups
  ORDER BY column      -- optional: define row order within each group
)
```

GROUP BY vs. Window Function

GROUP BY	Window Function
Collapses rows into one row per group	Keeps every original row
Can't show both detail and aggregate together	Can show a row's own value AND a group-level value side by side

```
-- Show each employee's salary alongside their department's average
SELECT name, department, salary,
       AVG(salary) OVER (PARTITION BY department) AS dept_avg
FROM employees;
```

Ranking Functions

Function	Behavior
<code>ROW_NUMBER()</code>	Unique sequential number per row — no ties (1, 2, 3, 4...)
<code>RANK()</code>	Same rank for ties, then skips numbers (1, 2, 2, 4...)
<code>DENSE_RANK()</code>	Same rank for ties, no skipping (1, 2, 2, 3...)
<code>NTILE(n)</code>	Divides rows into n roughly equal buckets

```
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
       RANK() OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk
FROM employees;
```

```
salary | row_num | rnk | dense_rnk
9000   | 1       | 1   | 1
9000   | 2       | 1   | 1    ← tie
```

8000	3	3	2	← RANK skips to 3, DENSE_RANK doesn't
7000	4	4	3	

Common Use Case: Top-N Per Group

```
-- Highest-paid employee in each department
SELECT * FROM (
    SELECT name, department, salary,
           ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rn
    FROM employees
) ranked
WHERE rn = 1;
```

Offset Functions

Function	Behavior
LAG(col, n)	Value from n rows before the current row
LEAD(col, n)	Value from n rows after the current row
FIRST_VALUE(col)	First value in the window
LAST_VALUE(col)	Last value in the window

```
-- Month-over-month sales comparison
SELECT month, sales,
       LAG(sales) OVER (ORDER BY month) AS prev_month_sales,
       sales - LAG(sales) OVER (ORDER BY month) AS change
FROM monthly_sales;
```

Running Totals

```
SELECT order_date, amount,
       SUM(amount) OVER (ORDER BY order_date) AS running_total
FROM orders;
```

Interview Q&A

Q1. Difference between RANK() and DENSE_RANK()?

Both assign the same rank to tied rows, but RANK() leaves a gap in the sequence after a tie (1,2,2,4), while DENSE_RANK() does not (1,2,2,3).

Q2. How would you find the top 3 highest-paid employees per department?

Use ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) in a subquery/CTE, then filter WHERE rn <= 3.

Q3. What does **PARTITION BY** do?

Splits rows into independent groups for the window function to operate on separately — similar to GROUP BY, but without collapsing rows.

Q4. How do you calculate a running total in SQL?

SUM(column) OVER (ORDER BY date_column) — the default frame accumulates from the start up to the current row.

INTERVIEW SUMMARY

- Window functions compute values across related rows without collapsing them — unlike GROUP BY.
- ROW_NUMBER(), RANK(), and DENSE_RANK() differ in how they handle ties.
- LAG()/LEAD() access neighboring rows; combined with ORDER BY, SUM() OVER() produces running totals.
- The ROW_NUMBER() + subquery pattern is the standard way to get "top-N per group."

Subqueries vs. CTEs (WITH Clause)

Both let you build queries out of smaller, nested pieces — but they differ in readability, reusability, and (on some engines) performance.

Subquery

A query nested inside another query — in `SELECT`, `FROM`, or `WHERE`.

```
-- Subquery in WHERE
SELECT name FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- Subquery in FROM (a "derived table")
SELECT dept, avg_salary FROM (
    SELECT department AS dept, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
) dept_avgs
WHERE avg_salary > 6000;
```

CTE (Common Table Expression)

A named, temporary result set defined with `WITH`, referenced later in the query — like a variable for a query.

```
WITH dept_avgs AS (
    SELECT department, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
)
SELECT * FROM dept_avgs
WHERE avg_salary > 6000;
```

Multiple / Chained CTEs

```
WITH high_earners AS (
    SELECT * FROM employees WHERE salary > 8000
),
by_dept AS (
    SELECT department, COUNT(*) AS cnt
    FROM high_earners
    GROUP BY department
)
SELECT * FROM by_dept ORDER BY cnt DESC;
```

Recursive CTE

A CTE that references itself — used for hierarchical data like org charts or category trees.

```
WITH RECURSIVE org_chart AS (  
    SELECT employee_id, name, manager_id, 1 AS level  
    FROM employees  
    WHERE manager_id IS NULL          -- anchor: top of the hierarchy  
    UNION ALL  
    SELECT e.employee_id, e.name, e.manager_id, oc.level + 1  
    FROM employees e  
    JOIN org_chart oc ON e.manager_id = oc.employee_id -- recursive step  
)  
SELECT * FROM org_chart;
```

Comparison Table

Feature	Subquery	CTE
Readability	Can get deeply nested and hard to read	Named, top-down, easier to follow
Reusability	Must repeat the same subquery to reuse it	Can be referenced multiple times in the same query
Recursion	Not supported	Supported (WITH RECURSIVE)
Scope	Local to where it's written	Available to the entire main query that follows
Performance	Engine-dependent; often inlined by the optimizer	Engine-dependent — some databases materialize CTEs, others inline them

NOTE ON PERFORMANCE

CTEs are *not* automatically faster or slower than subqueries — most modern query optimizers (PostgreSQL 12+, SQL Server, etc.) treat a non-recursive CTE the same as an equivalent subquery. Always check the execution plan rather than assuming.

Interview Q&A

Q1. When would you prefer a CTE over a subquery?

When the query needs to reference the same derived result multiple times, when you need recursion, or simply for readability in complex multi-step logic.

Q2. Can a CTE reference itself?

Yes, with WITH RECURSIVE — commonly used for hierarchical/tree-structured data like org charts or bill-of-materials.

Q3. Are CTEs always faster than subqueries?

No — performance depends on the specific database engine and optimizer; some materialize CTEs (compute once, store), others inline them just like subqueries.

INTERVIEW SUMMARY

- Subqueries nest a query inside SELECT/FROM/WHERE; CTEs name a temporary result set with WITH for reuse and readability.
- CTEs support recursion (WITH RECURSIVE); plain subqueries do not.
- Neither is inherently faster — check the actual execution plan on your specific database engine.

ACID Properties & Transactions

A **transaction** is a sequence of one or more SQL operations executed as a single logical unit of work — either all of it succeeds, or none of it does. **ACID** describes the guarantees a reliable database transaction must provide.

A – Atomicity
C – Consistency
I – Isolation
D – Durability

Atomicity

All operations in a transaction succeed together, or all are rolled back — there's no partial completion.

```
BEGIN TRANSACTION;  
UPDATE accounts SET balance = balance - 100 WHERE id = 1; -- debit  
UPDATE accounts SET balance = balance + 100 WHERE id = 2; -- credit  
COMMIT;  
-- If either UPDATE fails, ROLLBACK undoes both – money is never "lost".
```

Consistency

A transaction moves the database from one valid state to another, respecting all constraints, triggers, and rules (e.g. foreign keys, CHECK constraints). It never leaves data in a state that violates defined rules.

Isolation

Concurrent transactions don't interfere with each other — each transaction behaves as if it were running alone, even when many run simultaneously.

Isolation Level	Prevents
Read Uncommitted	Nothing — allows dirty reads
Read Committed	Dirty reads
Repeatable Read	Dirty reads + non-repeatable reads
Serializable	Dirty reads + non-repeatable reads + phantom reads (strictest)

Durability

Once a transaction is committed, it stays committed — even if the system crashes immediately afterward (typically achieved via write-ahead logging).

Transaction Control Statements

```
BEGIN TRANSACTION;  -- start
COMMIT;             -- save changes permanently
ROLLBACK;           -- undo changes since BEGIN
SAVEPOINT sp1;      -- create a rollback point within a transaction
ROLLBACK TO sp1;    -- undo back to a specific savepoint
```

Common Concurrency Problems

Problem	Description
Dirty Read	Reading uncommitted changes from another transaction that might later be rolled back
Non-Repeatable Read	Reading the same row twice in one transaction and getting different values, because another transaction committed a change in between
Phantom Read	Re-running the same query and getting a different set of rows, because another transaction inserted/deleted matching rows

Interview Q&A

Q1. What does ACID stand for?

Atomicity, Consistency, Isolation, Durability — the four guarantees of a reliable database transaction.

Q2. What is a dirty read?

When a transaction reads data written by another transaction that hasn't been committed yet — if that other transaction rolls back, the read data was never actually valid.

Q3. What's the strictest isolation level, and what's the trade-off?

Serializable — it prevents all concurrency anomalies (dirty, non-repeatable, and phantom reads) but at the cost of reduced concurrency/throughput, since transactions may block or retry more often.

Q4. How does atomicity relate to ROLLBACK?

Atomicity guarantees that if any part of a transaction fails, the whole transaction can be undone via ROLLBACK, leaving the database as if the transaction never ran.

INTERVIEW SUMMARY

- ACID = Atomicity, Consistency, Isolation, Durability — the core guarantees of relational transactions.
- Isolation levels trade off correctness guarantees (dirty/non-repeatable/phantom reads) against concurrency and performance.

- BEGIN / COMMIT / ROLLBACK / SAVEPOINT control transaction boundaries explicitly.

UNION vs. UNION ALL

Both combine the result sets of two or more `SELECT` statements vertically (stacking rows) — the difference is how duplicates are handled.

Requirements for Both

- Each `SELECT` must have the **same number of columns**
- Corresponding columns must have **compatible data types**
- Column names in the result come from the **first** `SELECT` statement

UNION

Combines results and **removes duplicate rows** — this requires an internal sort/dedup step.

```
SELECT city FROM customers
UNION
SELECT city FROM suppliers;
-- Duplicate city names appearing in both tables show up only once
```

UNION ALL

Combines results and **keeps all rows**, including duplicates — no dedup step, so it's faster.

```
SELECT city FROM customers
UNION ALL
SELECT city FROM suppliers;
-- Every row from both tables appears, duplicates included
```

Visual Comparison

	Table A: Cairo, Giza, Cairo	Table B: Giza, Alex
UNION	→ Cairo, Giza, Alex	(deduplicated)
UNION ALL	→ Cairo, Giza, Cairo, Giza, Alex	(everything kept)

Comparison Table

Feature	UNION	UNION ALL
Duplicates	Removed	Kept
Performance	Slower (requires a sort/dedup pass)	Faster (no dedup step)
Use When		

You need a distinct combined list

You know there's no overlap, or duplicates are meaningful/expected

PERFORMANCE TIP

If you know the two result sets can't overlap (e.g. combining data from two disjoint date ranges), always use UNION ALL — it skips the expensive deduplication step entirely.

Interview Q&A

Q1. When should you use UNION ALL instead of UNION?

Whenever duplicate rows are acceptable or impossible — UNION ALL avoids the deduplication overhead, making it faster on large result sets.

Q2. What are the requirements for combining two SELECTs with UNION?

Same number of columns, compatible data types in corresponding positions; the final column names come from the first SELECT.

INTERVIEW SUMMARY

- UNION deduplicates the combined result; UNION ALL keeps every row.
- UNION ALL is faster since it skips the sort/dedup step — prefer it when duplicates are known to be impossible or acceptable.

DELETE vs. TRUNCATE vs. DROP

All three remove data, but at very different levels — a classic interview question that tests whether you understand DML vs. DDL.

DELETE

A **DML** (Data Manipulation Language) statement that removes rows one at a time, logged individually — can be filtered with `WHERE` and rolled back.

```
DELETE FROM employees WHERE department = 'Sales';  
-- Removes only matching rows; can be rolled back within a transaction  
-- Table structure remains; auto-increment counters typically unaffected
```

TRUNCATE

A **DDL** (Data Definition Language) statement that removes **all** rows at once by deallocating data pages — much faster than DELETE, but cannot be filtered.

```
TRUNCATE TABLE employees;  
-- Removes ALL rows; cannot use WHERE  
-- Resets auto-increment counters; minimal logging; often auto-commits
```

DROP

A **DDL** statement that removes the **entire table** — data, structure, indexes, constraints, everything.

```
DROP TABLE employees;  
-- The table itself no longer exists — must be recreated from scratch to use again
```

Comparison Table

Feature	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
Removes	Selected rows (or all)	All rows	Entire table + structure
WHERE clause	Yes	No	N/A
Speed	Slower (row-by-row logging)	Fast (deallocates pages)	Fast
Rollback	Yes (within a transaction)	Depends on engine — often not, or limited	Depends on engine

Resets identity/auto-increment	No	Yes	N/A — table is gone
Table structure remains?	Yes	Yes	No
Triggers fire?	Yes	Usually no	N/A

Visual Summary

DELETE → removes rows (selectively), keeps table + structure
 TRUNCATE → removes ALL rows fast, keeps table + structure
 DROP → removes the table itself, including its structure

Interview Q&A

Q1. Why is TRUNCATE faster than DELETE?

DELETE logs each row removal individually (enabling rollback and trigger execution), while TRUNCATE deallocates entire data pages at once with minimal logging.

Q2. Can you roll back a TRUNCATE?

It depends on the database engine — some support it within a transaction, others auto-commit it immediately, making rollback impossible. Always check your specific engine's behavior.

Q3. If you want to keep the table but remove all its data and reset the auto-increment ID, which do you use?

TRUNCATE — it clears all rows and resets identity counters, while keeping the table structure intact.

INTERVIEW SUMMARY

- DELETE (DML): selective, row-logged, rollback-friendly, keeps structure.
- TRUNCATE (DDL): removes all rows fast, resets auto-increment, keeps structure, limited rollback support.
- DROP (DDL): removes the table entirely, including its structure and indexes.

SQL vs. NoSQL

A foundational data engineering question: when do you reach for a relational database versus a NoSQL store?

SQL (Relational) Databases

Structured, table-based storage with a fixed schema. Examples: PostgreSQL, MySQL, SQL Server, Oracle.

- Fixed schema, defined ahead of time
- Data organized into related tables with keys
- Strong ACID guarantees
- Powerful JOIN operations across tables
- Vertically scaled primarily (though modern systems support some horizontal scaling)

NoSQL Databases

Flexible-schema storage optimized for scale and specific access patterns. Major categories:

Type	Examples	Best For
Document	MongoDB, CouchDB	Semi-structured, nested/JSON-like data
Key-Value	Redis, DynamoDB	Simple, fast lookups by key (caching, sessions)
Column-Family	Cassandra, HBase	Massive write throughput, wide sparse tables
Graph	Neo4j, Amazon Neptune	Highly connected data (social networks, recommendations)

Comparison Table

Feature	SQL	NoSQL
Schema	Fixed, defined upfront	Flexible / dynamic
Data Model	Tables with rows and columns	Documents, key-value, wide-column, graph
Relationships	JOINS across normalized tables	Often denormalized/embedded to avoid joins
Consistency	Strong (ACID) by default	Often eventual consistency (tunable in many systems)
Scaling	Primarily vertical	Designed for horizontal scaling
Query Language	Standardized SQL	Varies by database (API/query language per system)
Best For	Structured data with complex relationships, transactions	Massive scale, flexible/evolving schemas, specific access patterns

When to Choose Which

CHOOSE SQL WHEN

Data is structured and relationships matter, you need strong transactional guarantees (banking, orders, inventory), or you need complex multi-table queries and reporting.

CHOOSE NOSQL WHEN

Schema will evolve frequently, you need to scale horizontally across many servers, data is naturally hierarchical/document-shaped (e.g. product catalogs), or you need extremely high write throughput.

Interview Q&A

Q1. What's the core trade-off between SQL and NoSQL?

SQL offers strong consistency, a fixed schema, and powerful relational querying via JOINS; NoSQL trades some of that structure/consistency for flexible schemas and easier horizontal scalability.

Q2. Give an example of when you'd pick a document database over a relational one.

When storing naturally nested, semi-structured data — like a product catalog with varying attributes per category — where forcing a rigid relational schema would require excessive joins or sparse columns.

Q3. What does "eventual consistency" mean?

A consistency model (common in many NoSQL systems) where, after a write, all nodes will reflect that write eventually — but reads immediately after a write may briefly return stale data.

INTERVIEW SUMMARY

- SQL databases: fixed schema, strong ACID consistency, relational JOINS, primarily vertical scaling.
- NoSQL databases: flexible schema, categories include document/key-value/column-family/graph, designed for horizontal scale, often eventual consistency.
- Choice depends on data shape, consistency needs, and scale requirements — not a strict "better/worse" comparison.